Optimization Example with Constraints and Lagrange Multipliers

Computational Models

Step-by-step resolution example of a constrained optimization problem using the **Lagrange Multipliers method** from two approaches: primal and dual approach.

1. Primal Problem (KKT Conditions). We simultaneously solve the entire system of equations describing the intersection point between the objective function and the constraint.
    a. We derive the Lagrangian function $\mathcal{L}(x, y, \lambda)$ with respect to all its variables (x, y, and λ) and set them to zero.
    b. We solve for the primal variables (x and y) in terms of the multiplier (λ), substitute them into the constraint equation to find the numerical value of λ^*, and finally obtain the optimal point (x^*, y^*). Interpretation: You tackle the problem directly by solving the simultaneous equation system that dictates the tangency point between the level curves of the objective function and the constraint.
2. Dual Problem (Minimization of the Dual Function). We decompose the problem into two parts (as indicated by the dual problem $\min_\lambda \max_{x,y} \mathcal{L}$):
    a. Inner maximization: analytically find how x and y behave as a function of λ by maximizing the Lagrangian, which allows you to construct the dual function $g(\lambda)$.
    b. Outer minimization: minimize the dual function $g(\lambda)$ with respect to λ to find the optimal multiplier λ^*.

1. Problem Definition

Objective function to maximize:

$$f(x, y) = 16 - (x^2 + y^2)$$

```
import numpy as np
import matplotlib.pyplot as plt


# Create 3D figure
fig = plt.figure(figsize=(13, 9))
ax = fig.add_subplot(111, projection='3d')

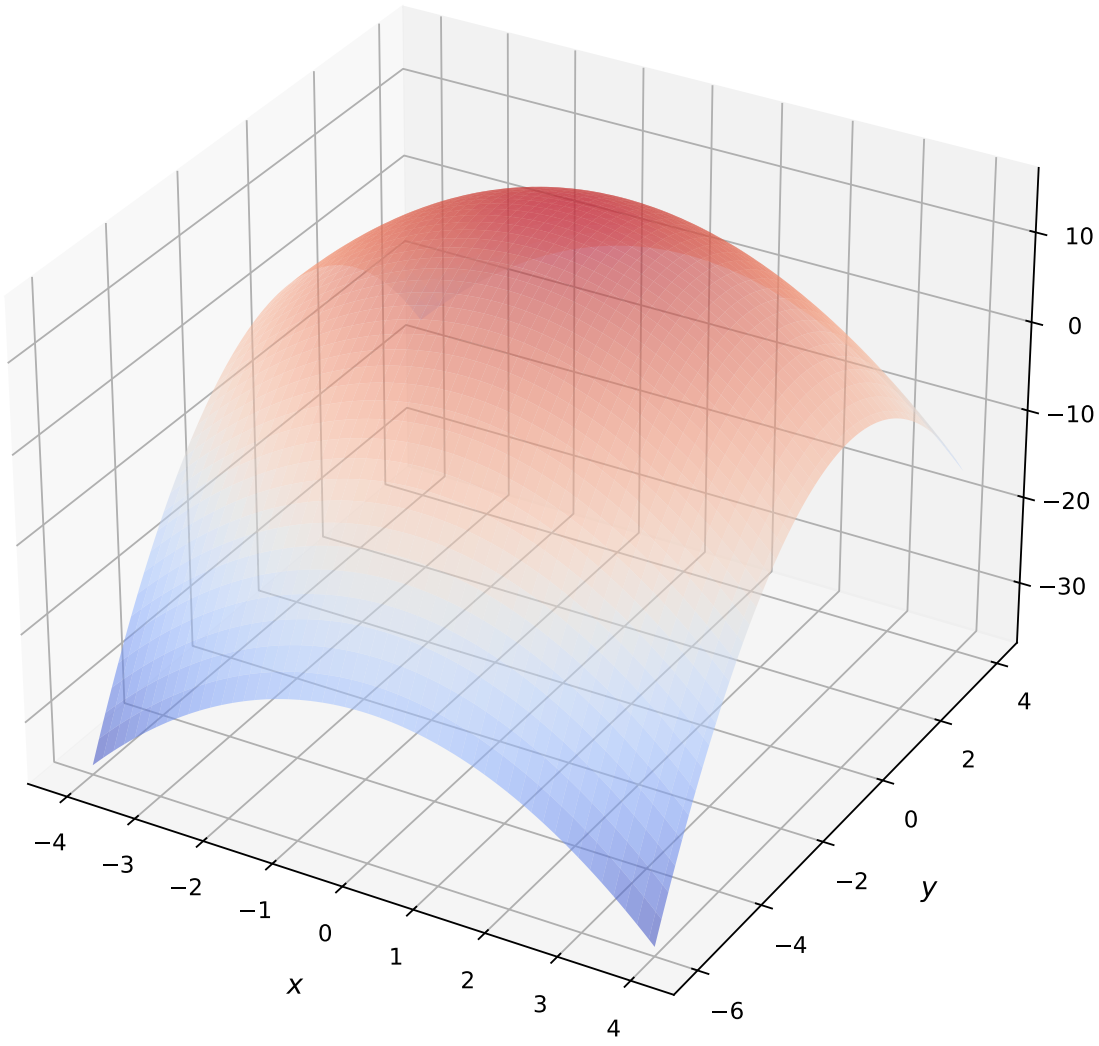
# 1. Full surface of the paraboloid f(x,y) = 16 - (x^2 + y^2)
x = np.linspace(-4, 4, 100)
y = np.linspace(-6, 4, 100)
X, Y = np.meshgrid(x, y)
Z = 16 - (X**2 + Y**2)

# Plot objective surface
surf = ax.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.55, edgecolor='none', antialiased=Tr

# Axis labels and formatting
ax.set_xlabel('$x$', fontsize=12, fontweight='bold', labelpad=10)
ax.set_ylabel('$y$', fontsize=12, fontweight='bold', labelpad=10)
ax.set_zlabel('$z=f(x, y)$', fontsize=12, fontweight='bold', labelpad=10)


# Add color bar and legend
#fig.colorbar(surf, shrink=0.4, aspect=5, label='Value of f(x,y)')
#ax.legend(loc='upper left', fontsize=10)

plt.show()
```



Constraint function:

$$g(x, y) = 2x - y = 4 \implies 2x - y - 4 = 0$$

```
import numpy as np
import matplotlib.pyplot as plt

# Create 3D figure
fig = plt.figure(figsize=(13, 9))
```

```
ax = fig.add_subplot(111, projection='3d')

# 1. Full surface of the paraboloid f(x,y) = 16 - (x^2 + y^2)
x = np.linspace(-4, 4, 100)
y = np.linspace(-6, 4, 100)
X, Y = np.meshgrid(x, y)
Z = 16 - (X**2 + Y**2)

# Plot objective surface
surf = ax.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.55, edgecolor='none', antialiased=T

# 2. Transparent vertical plane for the constraint: 2x - y = 4 => y = 2x - 4
# Since the plane is vertical (independent of z), we use a mesh in X and Z.
x_plane = np.linspace(-3, 4, 50)
z_plane = np.linspace(0, 16, 50)
X_pl, Z_pl = np.meshgrid(x_plane, z_plane)
Y_pl = 2 * X_pl - 4

# Plot transparent constraint plane
ax.plot_surface(X_pl, Y_pl, Z_pl, color='red', alpha=0.2, edgecolor='none', label='Constraint

# 3. Intersection curve (constraint projected onto the surface)
x_constr = np.linspace(-2.5, 3.5, 100)
y_constr = 2 * x_constr - 4
z_constr = 16 - (x_constr**2 + y_constr**2)
ax.plot(x_constr, y_constr, z_constr, color='darkred', linewidth=3.5, label='Intersection Cu

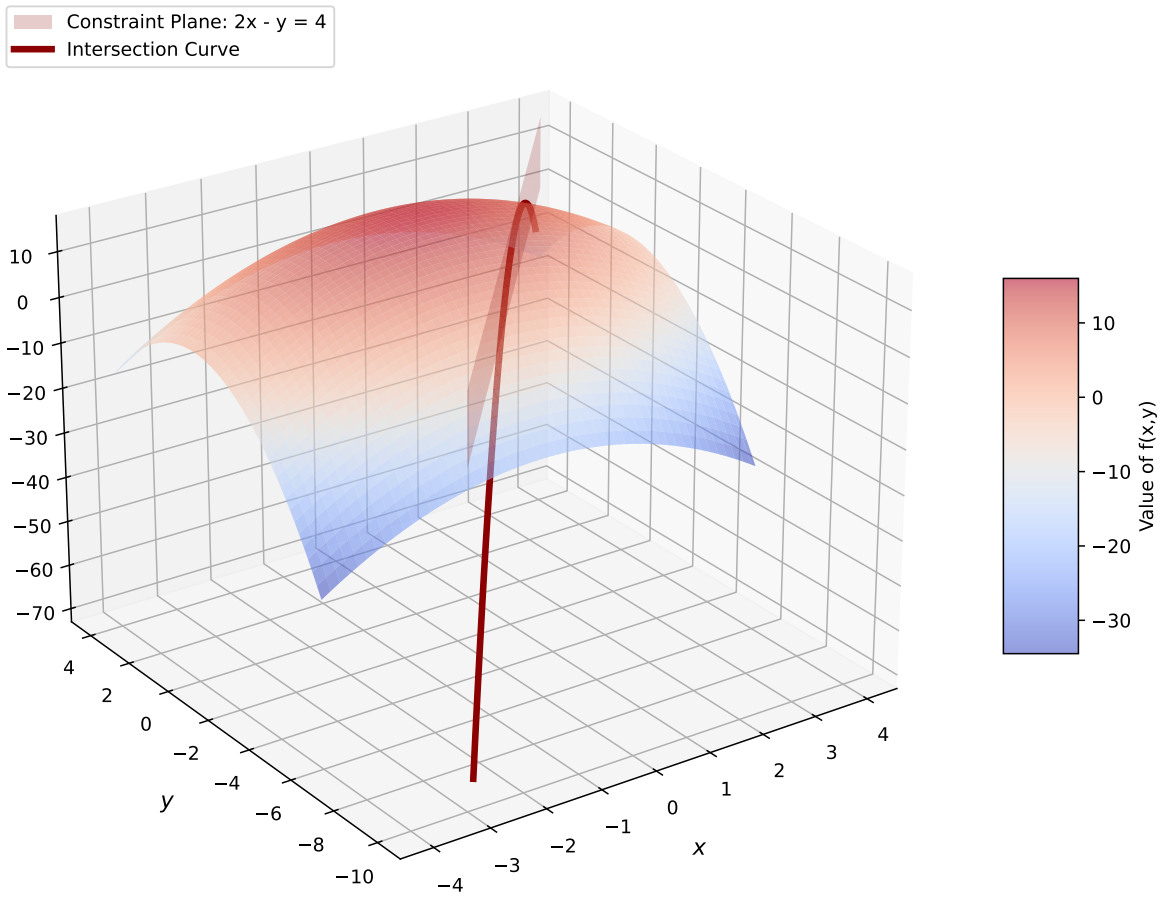
# Axis labels and formatting
ax.set_xlabel('$x$', fontsize=12, fontweight='bold', labelpad=10)
ax.set_ylabel('$y$', fontsize=12, fontweight='bold', labelpad=10)
ax.set_zlabel('$z=f(x, y)$', fontsize=12, fontweight='bold', labelpad=10)


#ax.set_title('Surface, Constraint Plane, and Optimal Point (Lagrange)', fontsize=14, fontwe


# Add color bar and legend
fig.colorbar(surf, shrink=0.4, aspect=5, label='Value of f(x,y)')
ax.legend(loc='upper left', fontsize=10)

# Adjust camera perspective to appreciate the plane cutting the paraboloid
ax.view_init(elev=25, azim=-125)
```

```
plt.show()
```



Lagrangian function:

$$\mathcal{L}(x, y, \lambda) = 16 - (x^2 + y^2) + \lambda(2x - y - 4)$$

2. Primal Solution via Lagrange Multipliers

To find the critical points, we set the partial derivatives of $\mathcal{L}(x, y, \lambda)$ to zero:

1. Derivative with respect to x:

$$\frac{\partial \mathcal{L}}{\partial x} = -2x + 2\lambda = 0 \implies x = \lambda \tag{1}$$

2. Derivative with respect to y:

$$\frac{\partial \mathcal{L}}{\partial y} = -2y - \lambda = 0 \implies y = -\frac{\lambda}{2} \tag{2}$$

3. Derivative with respect to λ (the original constraint):

$$\frac{\partial \mathcal{L}}{\partial \lambda} = 2x - y - 4 = 0 \tag{3}$$

Substitution to find λ, x, and y:

We substitute $x = \lambda$ and $y = -\frac{\lambda}{2}$ into the constraint from Equation 3:

$$2(\lambda) - \left(-\frac{\lambda}{2}\right) = 4$$

$$2\lambda + \frac{\lambda}{2} = 4$$

$$\frac{5\lambda}{2} = 4 \implies \lambda = \frac{8}{5} = 1.6$$

Now we calculate the optimal values for x and y in Equation 1 and Equation 2:

- $x^* = \frac{8}{5} = 1.6$
- $y^* = -\frac{1}{2} \cdot \left(\frac{8}{5}\right) = -\frac{4}{5} = -0.8$

3. Resolution from the Dual Perspective

The dual problem is formulated as:

$$\min_{\lambda} g(\lambda) \quad \text{where} \quad g(\lambda) = \max_{x,y} \mathcal{L}(x, y, \lambda)$$

First, find the dual function $g(\lambda) = \max_{x,y} \mathcal{L}(x, y, \lambda)$

We express x and y as functions of λ that maximize $\mathcal{L}$:

$$x(\lambda) = \lambda, \quad y(\lambda) = -\frac{\lambda}{2}$$

We substitute these values into the Lagrangian function $\mathcal{L}(x, y, \lambda)$:

$$g(\lambda) = 16 - \left(\lambda^2 + \left(-\frac{\lambda}{2}\right)^2\right) + \lambda\left(2\lambda - \left(-\frac{\lambda}{2}\right) - 4\right)$$

$$g(\lambda) = 16 - \left(\lambda^2 + \frac{\lambda^2}{4}\right) + \lambda\left(\frac{5\lambda}{2} - 4\right)$$

$$g(\lambda) = 16 - \frac{5\lambda^2}{4} + \frac{5\lambda^2}{2} - 4\lambda$$

$$g(\lambda) = 16 + \frac{5\lambda^2}{4} - 4\lambda$$

Second, minimize the dual function $g(\lambda)$ over λ

To find the minimum of $g(\lambda)$, we derive with respect to λ and set it to zero:

$$g'(\lambda) = \frac{5}{2}\lambda - 4 = 0 \implies \lambda^* = \frac{8}{5}$$

This gives us the same optimal values for $x^* = \frac{8}{5} = 1.6$ and $y^* = -\frac{4}{5} = -0.8$

Since $g''(\lambda) = \frac{5}{2} > 0$, this point represents a **strict minimum** for the dual function.

4. Final Result and Maximum Value

We evaluate the objective function $f(x, y)$ at the optimal point $(x^*, y^*) = \left(\frac{8}{5}, -\frac{4}{5}\right)$:

$$f\left(\frac{8}{5}, -\frac{4}{5}\right) = 16 - \left(\left(\frac{8}{5}\right)^2 + \left(-\frac{4}{5}\right)^2\right)$$

$$f\left(\frac{8}{5}, -\frac{4}{5}\right) = 16 - \left(\frac{64}{25} + \frac{16}{25}\right)$$

$$f\left(\frac{8}{5}, -\frac{4}{5}\right) = 16 - \frac{80}{25} = 16 - 3.2 = 12.8$$

Summary of the Solution and Why They Match:

- **Optimal Point**: $(x^*, y^*) = (1.6, -0.8)$ or $\left(\frac{8}{5}, -\frac{4}{5}\right)$
- **Optimal Lagrange Multiplier**: $\lambda^* = 1.6$ or $\frac{8}{5}$
- **Maximum Value Achieved**: $f(x^*, y^*) = 12.8$ or $\frac{64}{5}$

The primal and dual solutions coincide because the objective function is convex and the constraint is linear, which satisfies strong duality (the duality gap is zero).More information at https://en.wikipedia.org/wiki/Duality_(optimization)

Solution Visualization

```
import numpy as np
import matplotlib.pyplot as plt

# Create 3D figure
fig = plt.figure(figsize=(13, 9))
ax = fig.add_subplot(111, projection='3d')

# 1. Full surface of the paraboloid f(x,y) = 16 - (x^2 + y^2)
x = np.linspace(-4, 4, 100)
y = np.linspace(-6, 4, 100)
X, Y = np.meshgrid(x, y)
Z = 16 - (X**2 + Y**2)

# Plot objective surface
surf = ax.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.55, edgecolor='none', antialiased=T

# 2. Transparent vertical plane for the constraint: 2x - y = 4 => y = 2x - 4
# Since the plane is vertical (independent of z), we use a mesh in X and Z.
x_plane = np.linspace(-3, 4, 50)
z_plane = np.linspace(0, 16, 50)
X_pl, Z_pl = np.meshgrid(x_plane, z_plane)
Y_pl = 2 * X_pl - 4

# Plot transparent constraint plane
ax.plot_surface(X_pl, Y_pl, Z_pl, color='red', alpha=0.2, edgecolor='none', label='Constrain

# 3. Intersection curve (constraint projected onto the surface)
x_constr = np.linspace(-2.5, 3.5, 100)
y_constr = 2 * x_constr - 4
```

```
z_constr = 16 - (x_constr**2 + y_constr**2)
ax.plot(x_constr, y_constr, z_constr, color='darkred', linewidth=3.5, label='Intersection Cu
```

```
# 4. Optimal point found: (1.6, -0.8, 12.8)
x_opt = 1.6
y_opt = -0.8
z_opt = 16 - (x_opt**2 + y_opt**2)

# Mark the optimal point
ax.scatter([x_opt], [y_opt], [z_opt], color='gold', s=220, zorder=5, edgecolors='black', labe

# Dashed line from the optimal point to the XY plane for visual reference
ax.plot([x_opt, x_opt], [y_opt, y_opt], [0, z_opt], color='black', linestyle='--', linewidth=

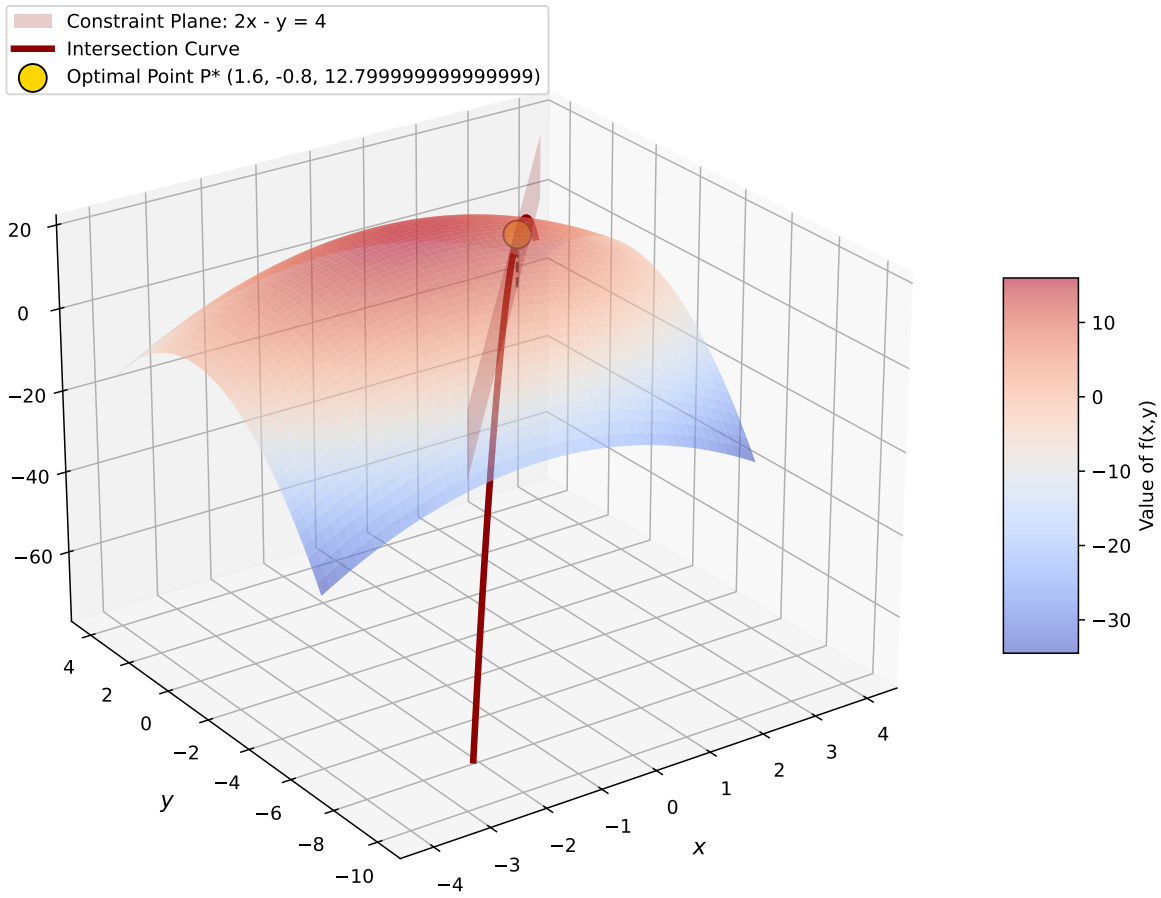
# Axis labels and formatting
ax.set_xlabel('$x$', fontsize=12, fontweight='bold', labelpad=10)
ax.set_ylabel('$y$', fontsize=12, fontweight='bold', labelpad=10)
ax.set_zlabel('$z=f(x, y)$', fontsize=12, fontweight='bold', labelpad=10)


#ax.set_title('Surface, Constraint Plane, and Optimal Point (Lagrange)', fontsize=14, fontwei

# Add color bar and legend
fig.colorbar(surf, shrink=0.4, aspect=5, label='Value of f(x,y)')
ax.legend(loc='upper left', fontsize=10)

# Adjust camera perspective to appreciate the plane cutting the paraboloid
ax.view_init(elev=25, azim=-125)

plt.show()
```


Constraint Plane: 2x - y = 4
Intersection Curve
Optimal Point P* (1.6, -0.8, 12.799999999999999)
20
0
−20
−40
−60
4
2
0
−2
−4
−6
−8
−10
y
−4
−3
−2
−1
0
1
2
3
4
x
10
0
−10
−20
−30
Value of f(x,y)